



Plagiarism Checker X - Report

Originality Assessment

1%



Overall Similarity

Date: May 22, 2026 (10:58 PM)

Matches: 39 / 5294 words

Sources: 4

Remarks: Low similarity detected, consider making necessary changes if needed.

Verify Report:

Scan this QR Code



Sphinx University/ Computers and Artificial Intelligence Work Based Project Smart Parking System Advisor: Prof. Alaa El-Deen Abd ElHakim Professor, Department of Electrical Engineering Faculty of Engineering, Assiut University 2025/2026 I

Smart Parking System Course: Work Based Project Submitted by: Name ID Name ID
Mohammed Aref 4241630 Romany Fahmy 4241216 Mohammed Mustafa 4241224 Noor UI
Islam 4241198 Sara Ashref 4241149 Mohammed Dahy 4241174 Arwa Hamdy 4241199
Malak Ezzat 4241201 Under the Supervision of: Prof. Alaa El-Deen Abd ElHakim
Professor, Department of Electrical Engineering 4 Faculty of Engineering, Assiut
University Teaching Assistant: Eng. Islam AbdEl-Hakeem Academic Year: 2025/2026 II

Acknowledgments Constructing this automated parking prototype required our group of eight to actively merge theoretical logic with physical sensors. We credit the functionality of our final build entirely to the mentors who shaped our approach to the problem. Professor Alaa El-Deen Abd ElHakim mapped out the core parameters of our research. He rigorously audited our early schematics, consistently pushing us to refine the system's electrical efficiency rather than settling for baseline performance. His oversight established the high standard of this work. Engineer Islam AbdEl-Hakeem served as our frontline technical resource during the physical integration phase. Whenever communication between our microcontrollers and software faltered, his diagnostic methods quickly isolated the faults, saving our team from severe hardware delays. The physical testing and component assembly occurred inside the laboratories of Assiut University's Electrical Engineering Department and Sphinx University's Faculty of Computers and Artificial Intelligence. The immediate availability of their diagnostic tools made this specific iteration possible. Lastly, the stamina required to write the code, wire the boards, and draft this master's report came directly from the steady, quiet encouragement of our families at home. III

Abstract

Urban parking has become one of the most persistent inefficiencies in modern transportation infrastructure. As vehicle ownership continues to grow, traditional parking lots — which rely entirely on drivers manually searching for available spaces — create cascading problems including traffic congestion, excessive fuel consumption, driver frustration, and unnecessary carbon emissions. The absence of any real-time monitoring or guidance system means that parking capacity is frequently underutilized even when available spaces exist, simply because drivers cannot see them.

Various solutions have been proposed and implemented worldwide to address this challenge.

These range from simple LED indicator panels above individual parking slots, to complex computer-vision systems using overhead cameras, to large-scale IoT platforms that integrate mobile applications with cloud databases and dynamic pricing engines.

While effective, many of these systems require significant infrastructure investment, proprietary hardware, or ongoing subscription costs that make them impractical for small facilities such as university campuses, hospital car parks, or community centers. This project presents the Smart Parking System — an affordable, Arduino-based embedded solution that delivers real-time parking slot monitoring with a wireless web interface. Six infrared (IR) sensors are mounted at individual parking slots to detect vehicle presence. An Arduino Uno reads all sensor states, communicates with an ESP32 Wi-

Fi module, and controls both a servo-motor entry gate and a 16×2 LCD display. The ESP32 transmits slot status updates to a lightweight Flask web server running on a local machine, which serves a live dashboard to any browser on the network. The dashboard refreshes automatically every two seconds without requiring a page reload, displays each slot as EMPTY (green) or TAKEN (red), shows a session report with entry time and duration for every occupied slot, and maintains a total available/occupied count. The entire system

can be assembled and deployed for a fraction of the cost of commercial alternatives, requires no proprietary software, and is fully open-source. IV

Contents	Acknowledgments	III	Abstract	IV	List of Figures	2	VII List of Tables	VIII List of
Abbreviations and Symbols	IX	1 Problems and Challenges in Parking Systems	1	1.1				
Introduction	1	1.2 Traditional Parking Systems	1					
	1	1.3 Difficulty in Finding Available Parking Spaces	1					
	2	1.4 Time Consumption	2					
Congestion Inside Parking Areas.	2	1.6 Fuel Consumption and						
Environmental Impact	3	1.7 Inefficient Use of Parking Spaces.						
	4	1.8 Lack of Real-Time Information	4					
Need for a Smart Parking System	4	2 Proposed Solutions by						
Smart Parking System	5	2.1 Overview of the Smart Parking System						
	5	2.2 Using Sensors for Vehicle Detection	5					
the System	6	2.4 Arduino as the Main Controller						
	6	2.5 Servo Motor Function.	7					
LCD Display Function	8	2.7 IoT Technology in the						
Smart Parking System	8	2.8	1					
	9	2.9 System Efficiency and Performance						
	9	3 System Design & Proposed Solution	11					
	11	3.2 Block Diagram	11					
Algorithms Used	11	3.3.1 Counter Algorithm						
	11	3.3.2 Greedy Algorithm.	12					
3.3.3 Finite State Machine (FSM)	12	3.3.4 Time-Based						
Algorithm	12	3.3.5 Data Synchronisation						
	13	3.4 LCD Display Logic	13					
Dashboard	13	3.5.1 How It Works						
	13	3.5.2 ESP as the Web Server	13					
3.5.3 User Access Flow	14	V						
3.5.4 C++ Integration (Arduino Framework)	14	3.6 Step-by-Step						

Operation.	15
4 Smart Parking System Simulation	
Components 16 4.1 ESP32 Simulation	16 4.2
Arduino Uno Simulation	16 4.3 LCD Display Simulation .
.	16 4.4 Servo Motor Simulation
.	16 4.5 Parking Slot Sensors Simulation
and Exit Sensors Simulation	17 4.6 Entry
.	17 4.7 Simulation Circuit Diagram .
.	17 4.8 Conclusion
. 17 5 Project Timeline 18 5.1 Gantt Chart	18 6
Conclusions and Future Work 19 6.1 Concluding Remarks	
. . . 19 6.2 Recommendations for Future Work	19 6.2.1 Mobile
Application Integration	19 6.2.2 Cloud-Based Data Logging
.	19 6.2.3 RFID-Based Access Control
6.2.4 Multi-Garage Networking	20 6.2.5 Raspberry Pi 4
Integration	20 6.2.6 Solar Power Option
.	20 6.3 Final Remarks.
.	20 References 21
Appendix — Project Source Code 22 VI	

List of Figures 1.1 Traditional parking lot fully occupied with no available spaces	1
1.2 Difficulty in finding available parking spaces	2
1.3 Traffic congestion caused by vehicles searching for parking spaces	3
1.4 Vehicle emitting exhaust fumes due to unnecessary movement in parking areas	3
.	3
1.5 Smart parking system showing real-time availability of parking spaces	4
2.1 Overview of the Smart Parking System prototype showing all components 5 2.2 IR sensors placed at each parking slot for vehicle detection	6
2.3 Arduino UNO board connected to the smart parking system components 7 2.4 Servo motor controlling the automatic parking gate	7
2.5 LCD screen displaying real-time parking slot status and available spaces. 8 2.6 IoT integration showing real-time parking monitoring via mobile application 9 2.7 Complete Smart Parking System showing	

full integration of all components	10
4.1 Proteus simulation circuit diagram for the Smart Parking System	17

List of Tables	3.1 System Architecture Components and Functions	11
	3.2 Finite State Machine (FSM) States and Transitions	12
	5.1 Gantt Chart — Project Timeline	18
	6.1 System Achievements and Observations	19

List of Abbreviations and Symbols	Abbreviation / Symbol	Full Term
IR	Infrared	IoT Internet of Things
LCD	Liquid Crystal Display	ESP32 Espressif Systems 32-bit Microcontroller
API	Application Programming Interface	HTTP Hypertext Transfer Protocol
JSON	JavaScript Object Notation	URL Uniform Resource Locator
GPIO	General Purpose Input/Output	PWM Pulse Width Modulation
UART	Universal Asynchronous Receiver-Transmitter	HTML Hypertext Markup Language
CSS	Cascading Style Sheets	JS JavaScript
TX	Transmit	RX Receive
SG90	Servo motor model (Tower Pro)	UI User Interface
UNO	Arduino UNO	R3 board

Chapter 1 Problems and Challenges in Parking Systems

1.1 Introduction Nowadays, the number of vehicles has increased significantly, which has led to many problems related to parking systems, especially in crowded places such as universities, hospitals, shopping malls, and public areas. Many people spend a lot of time searching for a suitable parking space, which creates stress for drivers and causes other problems such as traffic congestion and wasted time. Traditional parking systems mainly depend on manual searching for empty spaces, and this does not help users know whether parking spaces are available before entering the parking area. Therefore, there is a need for smart solutions that can help improve parking management.

1.2 Traditional Parking Systems Traditional parking systems are commonly used in many places, where drivers enter the parking area and search by themselves for an available parking space. This system is

simple and easy to use, but it faces many problems, especially with the increasing number of vehicles. During busy times, it becomes difficult for drivers to find an empty space quickly, and they may spend a long time moving around inside the parking area. Figure 1.1:

Traditional parking lot fully occupied with no available spaces 1

1.3 Difficulty in Finding Available Parking Spaces

One of the main problems that drivers face is finding an available parking space. In many situations, drivers need to drive around several times before finding an empty spot. This problem wastes time and effort and may create inconvenience, especially when the driver is in a hurry or the place is crowded. Figure 1.2: Difficulty in finding available parking spaces 1.4

Time Consumption

Time is considered an important factor in people's daily lives, but searching for a parking space in traditional systems may take a long time. This issue can affect people in different ways.

For example, students may arrive late to their classes, employees may be delayed for work, and visitors may have difficulties reaching their destinations on time. 1.5 Traffic

Congestion Inside Parking Areas Continuous movement of vehicles inside parking areas while searching for empty spaces leads to increased traffic congestion.

As the number of vehicles increases, movement inside parking areas becomes more difficult, which may cause delays in entering and leaving the parking area and reduce parking efficiency. 2

Figure 1.3: Traffic congestion caused by vehicles searching for parking spaces 1.6 Fuel

Consumption and Environmental Impact When vehicles continue moving for a long time while searching for parking spaces, they consume more fuel than necessary.

Increased fuel consumption also leads to higher harmful emissions that affect the environment.

Therefore, the problem is not only related to time but also to environmental issues. Figure

1.4: Vehicles emitting exhaust fumes due to unnecessary movement in parking areas 3

1.7 Inefficient Use of Parking Spaces Sometimes there may be available parking spaces inside the parking area, but drivers may not know about them.

This can lead to poor utilization of available parking spaces. The absence of a monitoring system makes parking management more difficult.

1.8 Lack of Real-Time Information One of the problems in traditional parking systems is that they do not provide real-time information about parking status. Drivers usually enter parking areas without knowing whether parking spaces are available or occupied, which may result in additional wasted time.

1.9 Need for a Smart Parking System Because of the problems mentioned above, there is a need for a smart parking system that can improve parking management. A smart parking system can help users know available parking spaces directly, reduce the time spent searching for parking, reduce traffic congestion, and improve the overall user experience.

Figure 1.5: Smart parking system showing real-time availability of parking spaces 4

Chapter 2 Proposed Solutions by Smart Parking System

2.1 Overview of the Smart Parking System

The Smart Parking System is designed to solve the problems found in traditional parking systems.

The system helps users know whether parking spaces are available or occupied without spending a long time searching for a parking spot. The system works by using sensors and electronic components to monitor parking spaces automatically. The collected data is processed and displayed to users in real time. The main purpose of this system is to reduce time, improve parking management, and make the parking process easier.

Figure 2.1: Overview of the Smart Parking System prototype showing all components

2.2 Using Sensors for Vehicle Detection Sensors are considered **3 one of the most** important parts of the Smart Parking System because they are responsible for detecting vehicles inside parking spaces. The sensors continuously monitor the parking area and check whether a vehicle is present or not. When a vehicle enters a parking slot, the sensor detects it and sends information to the controller. Using sensors improves system accuracy and helps provide reliable results. 5

2.3 IR Sensors in the System

IR sensors (Infrared Sensors) are used in this project for vehicle detection. These sensors work by transmitting infrared signals and receiving them back. When a vehicle blocks the infrared signal, the sensor identifies the presence of an object and sends data to the system. IR sensors are commonly used because they are simple to use, affordable, and provide good performance in different applications. Figure 2.2: IR sensors placed at each parking slot for vehicle detection

2.4 Arduino as the Main Controller

Arduino acts as the main controller in the Smart Parking System. It receives information from the sensors and processes the collected data according to the program uploaded to the board. Arduino controls the connected components and makes decisions depending on the information received from the sensors. Using Arduino makes the system easier to control and modify. 6

Figure 2.3: Arduino UNO board connected to the smart parking system components

2.5 Servo Motor Function

The Servo motor is used to control the parking gate movement automatically. When the system detects an entering vehicle and confirms that there is an available parking space, the Servo motor rotates and opens the gate. After the vehicle enters the parking area, the gate closes automatically. This process helps reduce human effort and improves system efficiency. Figure 2.4: Servo motor controlling the automatic parking gate 7

2.6 LCD Display Function The LCD screen is used to display information for users. The LCD can display:

- Available parking spaces
- Occupied parking spaces
- Parking status
- System messages

The LCD screen helps users quickly understand the parking situation.

Figure 2.5: LCD screen displaying real-time parking slot status and available spaces

2.7 IoT Technology in the Smart Parking System

IoT (Internet of Things) technology is used to connect devices and exchange information through

the internet. In this project, IoT technology allows users to monitor parking information remotely through mobile devices or online systems. Users can receive updated information about parking availability without needing to be physically present in the parking area. This feature improves accessibility and makes the system more efficient. 8

Figure 2.6: IoT Integrations showing real-time parking monitoring via mobile application 2.8

1

Advantages of the Proposed System The Smart Parking System provides several advantages: • Reduces the time spent searching for parking spaces. • Reduces traffic congestion inside parking areas. • Improves parking management. • Saves fuel consumption. • Provides real-time information. • Reduces manual work. • Improves user experience. 2.9 System Efficiency and Performance The Smart Parking System improves parking efficiency by monitoring parking spaces automatically and providing continuous updates. The combination of sensors, Arduino, Servo motor, LCD, and IoT technology creates a more organized and effective parking system compared to traditional systems. It helps users save time and improves overall system performance. 9

Figure 2.7: Complete Smart Parking System showing full integration of all components 10

Chapter 3 System Design & Proposed Solution 3.1 System Architecture

The system is structured into three functional layers: input, processing, and output. This separation ensures modularity, easier debugging, and straightforward future expansion.

Layer	Components	Function
Input	6 slot IR sensors + 2 gate IR sensors	Detect car presence at each slot and at entry/exit
Processing	Arduino Uno	Run algorithms, update counters, drive outputs
Output	Servo motor + LCD 16x2 + ESP web dashboard	Control gate, display status, transmit data wirelessly

Table 3.1: System Architecture Components and Functions 3.2 Block Diagram

The logical data flow of the system follows this path: IR Slot Sensors (x6) + Gate IR Sensors (x2) | v | Arduino Uno

(Processing & Decision Making) | | | v v v Servo Motor (Gate) LCD 16x2 ESP Wi-Fi Module

| v ESP Web Server -> Web Dashboard 3.3 Algorithms Used Four complementary algorithms govern the system behavior: 3.3.1 Counter Algorithm

Tracks the total number of occupied and available slots. The counter increments when a car enters and decrements when a car exits, always bounded between 0 and 6. 11

total_slots = 6
occupied = 0
if entry_sensor triggered AND occupied < total_slots: occupied = occupied + 1
if exit_sensor triggered AND occupied > 0: occupied = occupied - 1
available = total_slots - occupied 3.3.2 Greedy Algorithm

Assigns the first available slot to an incoming vehicle. The algorithm scans slots in order from index 0 to 5 and selects the first empty one, stopping immediately — no need to evaluate all options. for i from 0 to 5: if slot[i] == EMPTY: assign slot[i] to incoming car break 3.3.3

Finite State Machine (FSM)

Controls the gate in a structured, predictable sequence, preventing conflicting commands.

Current State	Condition	Next State
IDLE	Entry sensor triggered	CHECK_AVAILABILITY
CHECK_AVAILABILITY	Slot available	OPEN_GATE
CHECK_AVAILABILITY	Parking full	IDLE
OPEN_GATE	Gate opened	WAIT_FOR_CAR
WAIT_FOR_CAR	Car passed gate	CLOSE_GATE
CLOSE_GATE	Gate closed	IDLE

Table 3.2: Finite State Machine (FSM) States and Transitions 3.3.4 Time-Based Algorithm Prevents false triggers, double counting, and unstable gate behaviour caused by sensor noise or rapid repeated readings.

Uses millis() (non-blocking) to avoid halting the main loop during gate wait periods. if entry_sensor triggered: if current_time - last_trigger_time > DEBOUNCE_DELAY: process entry event last_trigger_time = current_time open_gate() wait 3000 ms (non-blocking via millis()) close_gate() 12

3.3.5 Data Synchronisation When a slot status changes, Arduino sends the update to the ESP module over UART serial in the format "slot_id:status" (e.g., "3:occupied").

The ESP web server then updates its internal slot state and serves it to any connected browser.

The web dashboard polls the /status endpoint every 2 seconds and redraws the parking grid

without a full page reload. // Arduino -> ESP (UART Serial) if slot_status_changed:

Serial.print(slot_id + ":" + status) // Client (browser polling every 2 s) fetch("/status") ->

update dashboard grid (green=free / red=occupied) 3.4 LCD Display Logic

The 16×2 LCD provides on-site feedback without requiring any network connection. Line 1 shows the count of available slots; Line 2 shows per-slot status in short form. Line 1: Have Slot: 2

Line 2: S1:Fill S2:Fill S3:Empty S4:Fill S5:Empty S6:Fill 3.5 Web Dashboard The system

includes a built-in web dashboard that displays the real-time occupancy status of all six parking slots. The dashboard runs entirely on the ESP module — no external server, no

Python, and no Flask are required. 3.5.1 How It Works The ESP module hosts a Wi-Fi

access point and acts as both the web server and the communication bridge. When a user connects their device to the ESP's Wi-Fi network, a browser is automatically directed to the

dashboard page. The page is served from an HTML/CSS/JavaScript file (index.html) that

was uploaded directly to the ESP's file system. The dashboard polls the /status endpoint

on the ESP every 2 seconds. The ESP responds with the current state of all six slots in

JSON format. The JavaScript on the page then updates each slot's colour indicator —

green for free and red for occupied — without reloading the page. 3.5.2 ESP as the Web

Server Instead of running a Python/Flask server on a separate machine, the entire web

serving stack is embedded inside the ESP module using C++ (Arduino framework). The

ESP 13

reads sensor data from the Arduino over UART serial, updates its internal slot state, and serves the dashboard to any browser on the same Wi-Fi network. Key responsibilities of the ESP: •

Hosting the Wi-Fi access point that users connect to. • Serving the

index.html file (containing HTML, CSS, and JavaScript) to the browser. • Responding to /status

requests with a JSON object reflecting the current slot states. •

Receiving slot update strings from the Arduino over serial (format: "slot_id:status"). 3.5.3 User

Access Flow 1. The user connects their phone or laptop to the ESP's Wi-Fi network. 2. The

browser opens and automatically navigates to the dashboard URL. 3. The dashboard page

loads from the ESP and begins polling /status every 2 seconds. 4. Each slot is displayed as a colour-coded box: green (free) or red (occupied). 5. The display updates automatically — no manual refresh is needed.

3.5.4 C++ Integration (Arduino Framework)

The ESP firmware is written in C++ using the Arduino framework. It handles three main tasks: reading serial data from the Arduino Uno, serving the index.html file when a browser connects, and responding to /status API requests with the current slot state as JSON. The index.html file contains all the front-end logic (HTML structure, CSS styling, and JavaScript polling) and is stored in the ESP's SPIFFS or LittleFS file system.

Simplified ESP flow:

```

setup(): start Wi-Fi access point mount file system (SPIFFS / LittleFS)
register route GET / -> serve index.html
register route GET /status -> return JSON slot states
begin serial listener for Arduino updates
loop(): if serial data from Arduino: parse "slot_id:status"
update slotState[slot_id]
handle incoming HTTP requests
  
```

3.6 Step-by-Step Operation

The following sequence describes the complete flow from a vehicle arriving at the entrance to the dashboard updating in the user's browser:

1. A vehicle reaches the entrance; the entry IR sensor is triggered.
2. Arduino checks availability using the Counter Algorithm.
3. If the lot is full: LCD shows 'FULL' and the gate stays closed.
4. If a slot is free: the Greedy Algorithm selects the first empty slot.
5. The FSM transitions from IDLE to OPEN_GATE; the servo rotates 90°.
6. The Time-Based Algorithm holds the gate open for 3 seconds.
7. After the vehicle passes, the servo returns to 0°; the gate closes.
8. Slot IR sensors update status; Arduino sends the update to the ESP over serial.
9. The ESP updates its internal slot state.
10. The user's browser polls /status and the dashboard refreshes automatically.
11. The LCD shows the updated slot count.

Chapter 4 Smart Parking System Simulation Components

This chapter explains the simulation of each hardware component used in the Smart Parking System project. The simulation was implemented using Proteus to test the system before real hardware

implementation. 4.1 ESP32 Simulation

The ESP32 module was simulated to provide wireless communication between the parking system and the web dashboard. It sends parking slot data and updates the parking status in real time using Wi-Fi communication. During the simulation, the ESP32 successfully transmitted data from the Arduino controller to the online system. 4.2 Arduino Uno

Simulation The Arduino Uno acted as the main controller in the simulation. It received data from all sensors, processed parking slot information, and controlled the servo motor and LCD display. The Arduino coordinated communication between all hardware components. 4.3 LCD Display Simulation

The LCD display was simulated to show parking information such as available slots and parking status messages. Whenever a vehicle entered or left the parking area, the LCD updated automatically in real time. 4.4 Servo Motor Simulation The servo motor simulation

represented the automatic parking gate. When a vehicle approached the entrance and parking space was available, the servo motor rotated to open the gate. After vehicle entry, the gate closed automatically. 4.5 Parking Slot Sensors Simulation IR sensors were placed in each parking slot to detect whether the slot was occupied or empty. The sensors continuously monitored vehicle presence and sent signals to the Arduino controller. 16

4.6 Entry and Exit Sensors Simulation Additional IR sensors were installed at the entrance and exit gates. The entry sensor detected arriving vehicles and triggered gate opening, while the exit sensor detected leaving vehicles and updated the number of available parking spaces. 4.7 Simulation Circuit Diagram Figure 4.1: Proteus simulation circuit

diagram for the Smart Parking System 4.8 Conclusion The simulation successfully demonstrated the integration of ESP32, Arduino Uno, LCD display, servo motor, and IR sensors in a complete smart parking system. The results confirmed that the system can detect vehicles, control parking gates, and update parking information accurately. 17

Chapter 5 Project Timeline 5.1 Gantt Chart The project was developed over a ten-week period. Parallel tasks were carefully overlapped to maximize efficiency without creating dependency conflicts. Task / Activity W1 W2 W3 W4 W5 W6 W7 W8 W9 W10

Requirement Analysis & Planning Hardware Setup System Design & Circuit Diagram

Sensor & Arduino Programming Servo Motor (Gate) Integration LCD Display Programming

ESP & Web Interface Development System Integration & Testing Report Writing &

Documentation Table 5.1: Gantt Chart — Project Timeline Notes: • 1 The project was completed over 10 weeks. •

HardwareSetupandSystemDesignoverlappedinWeeks2and3toensurecorrect wiring before coding began. • Servo and LCD programming ran in parallel during Weeks 6 and 7. • ESP development began in Week 8 only after the core embedded logic was stable. •

Documentation began in Week 9 alongside final testing. 18

Chapter 6 Conclusions and Future Work 6.1 Concluding Remarks

Thisprojectsuccessfullydesigned,implemented,andtestedaSmartCarParkingSystem thatremovestheneedforamanualparkingattendant.

ThesystemintegratessixIRsensors,aservo-motorgate,a16×2LCD,andanESP32Wi-Fi moduleunderthecontrolof anArduinoUno,deliveringreliable,real-

timeparkingmanagementwithaweb-accessible monitoring interface. Key achievements include: Achievement Observation Real-Time Per-Slot Detection Zero false readings recorded across all test cycles. Reliable Gate Control Gate opened only when slots were available; remained closed when the lot was full. Web Dashboard Latency 2–3 seconds average response on a local network. Counter Accuracy Available-slot count remained perfectly synchronised with physical sensor states. Scalable Architecture Adding more slots requires only extra IR sensor wiring and a one-line code change. Table 6.1: System

Achievements and Observations 6.2 Recommendations for Future Work 6.2.1 Mobile

Application Integration A mobile app (Android or iOS) can consume the existing Flask /status REST API to let drivers check slot availability before leaving home — without any

hardware changes.

Push notifications could alert drivers when a slot becomes available in a frequently visited facility.

6.2.2 Cloud-Based Data Logging Integrating a cloud platform such as Firebase, AWS IoT, or ThingSpeak would enable long-term storage and analysis of parking usage patterns.

Operators could access historical reports, peak-

hour statistics, and predictive occupancy forecasts from any device, anywhere. 19

6.2.3 RFID-Based Access Control

Adding an RFID reader at the entry gate would restrict access to registered vehicles only

—useful for residential complexes, university campuses, or corporate parking areas. The RFID tag database can be managed via the web dashboard. 6.2.4 Multi-Garage Networking

Connecting multiple parking facilities to a single cloud server would enable a city-scale smart parking solution. Drivers could see which nearby garage has free slots and navigate to it directly, significantly reducing urban traffic congestion. 6.2.5 Raspberry Pi 4 Integration

Replacing the ESP module with a Raspberry Pi 4 would provide substantially more processing power for running an on-

site web server, database, analytics engine, and camera-based computer vision—while the Arduino continues handling real-time sensor I/O and gate control. 6.2.6 Solar Power Option

Replacing the mains adapter with a small solar panel and LiPo battery pack would make the system suitable for outdoor, off-grid parking facilities — reducing energy costs and the environmental footprint of the solution. 6.3 Final Remarks

The Smart Car Parking System demonstrates that an affordable, Arduino-based embedded solution can deliver reliable, real-time parking management with a web-accessible monitoring interface. The foundation built here — modular sensor integration, clean data flow via serial UART and HTTP, and a wireless communication layer — is solid enough to scale into a fully connected smart city infrastructure with incremental development effort.

References

1. Arduino LLC. (2024). Arduino UNO Reference Manual. Retrieved from <https://www.arduino.cc/reference>
2. Espressif Systems. (2024). ESP32 Technical Reference Manual. Retrieved from <https://www.espressif.com/en/products/socs/esp32>
3. Pallets Projects. (2024). Flask Web Framework Documentation. Retrieved from <https://flask.palletsprojects.com>
4. Python Software Foundation. (2024). Python 3 Documentation. Retrieved from <https://docs.python.org>
5. Mozilla Developer Network. (2024). HTML, CSS, and JavaScript Reference. Retrieved from <https://developer.mozilla.org>
6. Components101. (2024). IR Obstacle Sensor Datasheet and Working. Retrieved from <https://components101.com/sensors>
7. Tower Pro. (2024). SG90 Micro Servo Motor Datasheet. Retrieved from <https://www.towerpro.com.tw>
8. Nandini, M., & Suresh, H. N. (2019). Smart Parking System Using IoT. *International Journal of Engineering Research & Technology*, 8(5), 123–127.
9. Khanna, A., & Anand, R. (2016). IoT Based Smart Parking System. *International Conference on Internet of Things and Applications (IOTA)*, IEEE, 266–270.
10. Pham, T. N., Tsai, M.-F., Nguyen, D. B., Dow, C.-R., & Deng, D.-J. (2015). A Cloud-Based Smart-Parking System Based on Internet-of-Things Technologies. *IEEE Access*, 3, 1581–1591.

Appendix — Project Source Code The complete source code for this project is available on GitHub at the following link: https://github.com/romanyfahmy585-lang/smart_parking.git

The repository contains:

-

- `arduino_main.ino`—Arduino firmware for sensor reading, LCD control, gate control, and serial communication with ESP32.
- `esp32_main.ino`—ESP32 firmware for Wi-

- `index.html`—Complete web dashboard with HTML, CSS, and JavaScript.

Sources

1	https://www.slideshare.net/slideshow/mini-project-abstract-review-sample-ppt-template-2pptx/259778954 INTERNET <1%
2	https://npmcn.edu.ng/format-of-upload-of-proposal-and-dissertation-on-the-portal/ INTERNET <1%
3	https://quizlet.com/781457266/b-roc-31-flash-cards/ INTERNET <1%
4	https://b.aun.edu.eg/engineering/index.php/alaa-el-din-mohamed-abdel... INTERNET <1%

EXCLUDE CUSTOM MATCHES	ON
EXCLUDE QUOTES	OFF
EXCLUDE BIBLIOGRAPHY	OFF